Research on Anomaly Detection Based on System Log Data

Baojia Zhang
School of Software
Shenyang University of Technology
Shenyang, Liaoning, China
2474986199@qq.com

Yan Jiang*
School of Software
Shenyang University of Technology
Shenyang, Liaoning, China
jiangyansygd@sut.edu.cn

Abstract

As modern software scales up and becomes increasingly complex, using log files to detect anomalies and ensure software reliability has become a research hotspot. However, the accuracy of existing log anomaly detection algorithms may decrease with system updates. In this paper, we propose a log anomaly detection model called LogCGA. First, we use the Drain algorithm to extract log templates, then divide log events into log sequences using fixed and sliding windows. We then extract statistical features and semantic features from the log using template counters and FastText language models, respectively. Finally, we input the feature vector into a parallel CB model for detection. We conduct experiments on public datasets and show that LogCGA significantly improves the accuracy and stability of anomaly detection compared to benchmark models.

CCS Concepts

• **Software and its engineering** → Software creation and management; Software verification and validation.

Keywords

Anomaly Detection, Log analysis, Word embedding, neural network, Deep learning, BiGRU, Attention Mechanism

ACM Reference Format:

Baojia Zhang and Yan Jiang. 2024. Research on Anomaly Detection Based on System Log Data. In *2024 5th International Conference on Computer Science and Management Technology (ICCSMT 2024), October 18–20, 2024, Xiamen, China.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3708036.3708050

1 Introduction

In recent years, with the rapid development of the internet industry, the scale of computer software systems has been continuously expanding, and their architectures have become increasingly complex. Especially in the dynamic and complex environments of real life, systems inevitably encounter various types of failures during operation. However, modern software systems are typically composed of numerous service components and involve highly intricate operational logic, which makes identifying and troubleshooting anomalies particularly difficult. If abnormal events are not promptly and effectively addressed, they can lead to a decline in system performance, negatively impact user experience, and even cause significant financial losses. Additionally, in high-risk industries such as finance, healthcare, or aviation, system failures can have even more severe consequences. Therefore, the ability to quickly and accurately detect system anomalies and take timely corrective actions is crucial for ensuring the stability and security of large-scale complex systems. The research and deployment of advanced anomaly detection technologies have become a key factor in ensuring the reliable operation of large-scale systems.

Log data records detailed information and behavior during system operation. Therefore, in traditional methods, when a system anomaly occurs, maintenance personnel can troubleshoot by analyzing the logs to ensure system reliability. Operations personnel manually analyze logs by matching regular expressions or searching keywords such as' failure '. However, the current large-scale systems are massive in scale and generate an enormous amount of log data every day. For example, commercial cloud programs can generate one billion bytes of data per hour[1]. The traditional approach to dealing with such massive logs is time-consuming and prone to errors. Therefore, researchers propose a data-driven approach to automatically detect anomalies by analyzing log data.

The existing methods for automated log detection include machine learning and deep learning, which aim to improve the performance of log anomaly detection. For example, Li[3] used a deep time convolutional network to extract fine-grained features from log events, and developed a new federated learning framework to support Internet devices to establish a more comprehensive anomaly detection model. Yang[2] proposed a semi supervised learning framework PLELog based on probabilistic label estimation, which uses clustering method HDBSCAN to estimate the probability of unlabeled log sequence labels, solving the problem of insufficient labels; And designed a gated recurrent unit neural network based on attention mechanism for anomaly detection. In addition, Duan[4] used a value function based reinforcement learning method Q-learnin to construct an anomaly detection model, which learns a directed acyclic graph of log patterns from normal execution, solves the Q value of each log execution path, and uses the Q value to determine whether the log sequence is abnormal. These methods have promoted the improvement of detection performance.

At present, log anomaly detection still faces many problems. Firstly, most of the time the computer system is in a normal state, with relatively few abnormal states, resulting in uneven distribution of log data and causing an imbalance in log data. Secondly, over time, requirements often change, and log statements also change accordingly. The instability of this data leads to a decrease in the

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCSMT 2024, October 18–20, 2024, Xiamen, China
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0999-9/24/10
https://doi.org/10.1145/3708036.3708050

accuracy of anomaly detection. To address these issues. Propose the LogCGA model, which includes three stages: log parsing, feature extraction, and anomaly detection. As shown in Figure 1, it has been experimentally proven that the proposed model improves the performance of detection.

2 Methods

2.1 Log parsing

Logs are usually unstructured textual data, and the purpose of log parsing is to structure the log data, extract log templates and parameters. The existing log parsing methods can be divided into three categories based on data mining techniques: frequent pattern mining, heuristics, and clustering. For example, LOgram [6] uses a dictionary to store the frequency of n-grams in logs, distinguishes templates and parameters, and achieves efficient log parsing. SHISO[7] clusters based on character types and vector information. Accurately extracted templates. Heuristic methods assume that log data contains features that ordinary text does not have. The Drain algorithm utilizes these unique features to make it superior to other techniques in terms of time efficiency and accuracy. Therefore, this article uses the Drain algorithm for log parsing. The Drain algorithm maintains logs by constructing a fixed depth parse tree, where leaf nodes represent a collection of log event types. When a new log appears, the new log first counts the number of words, and the same number is classified into the same leaf node. Otherwise. create a new leaf node. Then.compare the similarity with the log template set in the leaf node. If it is greater than the set threshold, it will be classified as the same template. Otherwise, create a new template. The similarity calculation formula is as follows (Formula 1)

$$Sim\,(seq1, seq2) = \frac{\sum_{i=1}^{n} eq(seq1\,(i)\,, seq2\,(i)}{n} \tag{1}$$

eq is a function that compares two words to see if they are the same. The value of Sim represents the similarity between two statements. After parsing the log, the leaf nodes in the tree are traversed to obtain the log template mi=[m_1, m_2, m_3,..., m_i].

A log sequence is a chronological arrangement of log entries generated during the operation of a system, recording various operations and events of the system or application. The log anomaly detection in this article is analyzed from a sequence perspective, and the incorrect execution order and incomplete execution of log events can lead to system anomalies. Therefore, after the log parsing is completed, the log events are grouped into log sequences seq={L_1, L_2,..., L_n} based on sliding windows and fixed windows. When it is determined that Li in seq is an abnormal log, then the log sequence is an abnormal sequence.

2.2 Feature extraction

2.2.1 Statistical feature extraction. The system program is executed according to a specific process, and there is a constant behavioral relationship between various log events during the execution process. For example, the number of logs for "open file" and "close file" should be equal under normal circumstances, and the number of log events should be relatively stable over a period of time. Statistically analyze this quantitative relationship, if during a certain period of time, this quantitative relationship does not hold or the number of logs increases dramatically. It indicates that the system may have encountered an exception. To capture this relationship, this article designs a log event counter. Specifically, assuming a dataset contains four templates represented as {m1, m2, m3, m4}, for a log sequence seq={L1, L2, L2, L3, L1, L4}, the corresponding log count vector is V=[v (L1), v (L2), v (L3), v (L4)], i.e. [1, 1, 2, 2], where v (Li) is the number of times the log template mi appears.

2.2.2 Semantic feature extraction. As systems are updated and upgraded, the syntax and format of logs may change, such as field name adjustments or log structure reorganization, which can lead to a decline in the accuracy of detection models. To address this issue, this paper extracts semantic features from the logs, ensuring that even when the log content changes, the core semantics of abnormal events remain consistent. Different versions of a system may use different words to describe the same operations or anomalies, but by extracting these semantic features, the model can still effectively identify abnormal behavior based on semantic consistency and capture hidden patterns in the logs. Currently, there are many methods available to extract semantic features from logs. For example, Word2Vec[8] or GloVe can be used to extract word vectors, but these two models can only represent words in the lexicon as vectors and cannot handle long words that have not been logged in and deformations of the same word. Therefore, this article uses the FastText algorithm to train word vectors, considering local word order features, processing long words and unregistered words, so that different forms of the same word have similar word vectors. Capture the intrinsic relationships between words in natural language, use n-gram method, consider character level information, generate vectors to represent the corpus, and use a hierarchical softmax linear classifier to calculate the probability distribution. The loss function is (Formula 2)

$$L = -\frac{1}{Num} \sum_{i=1}^{Num} y_i \log\left(f\left(BAx_i\right)\right) \tag{2}$$

Among them: Num is the number of samples; X_i is the regularized feature of the i-th sample, and y_i is the feature label corresponding to the i-th sample; A and B are weight matrices; F is the softmax function.

After obtaining the embedding vector of the word, in order to further highlight the important words in the sentence. The TF-IWF algorithm is used to represent the relationship between words and log events. The TF (Formula 3) word frequency matrix measures the association between a word and a given text, while the IWF measures the importance of the word. The calculation formula is as follows (Formula 4):

$$TF\left(word_i\right) = Count\left(\frac{word_i}{Total\left(word\right)}\right) \tag{3}$$

$$IWF\left(word_i\right) = \log\left(\frac{Total\left(event\right)}{Total\left(event_i\right)}\right) \tag{4}$$

Treat all templates as a corpus, where: Count ($word_i$) refers to the number of times the word $word_i$ appears in a certain log event; Total (word) refers to the number of words in the log event; Total (event) refers to the sum of the frequencies of all words in the corpus. Total ($event_i$) refers to the total number of times the

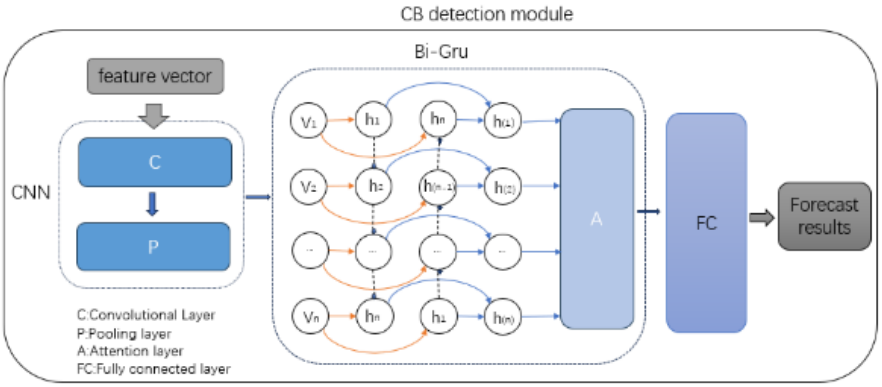

Figure 1: Framwork of CB detection module.

word $word_i$ appears in the corpus. From this, it can be seen that the weight of words in the log increases proportionally with the number of words, but if the word appears too many times in the entire corpus, its importance will decrease.

After calculating the weight Wi of each word, using the word vectors Vi of n words in the log event, the sentence embedding vector V of the entire log event is obtained by weighted summation. The formula is (Formula 5).

$$V = \sum_{i=1}^{n} W_i V_i \tag{5}$$

This method ensures that the sentence embedding vectors of each log event have consistent dimensions, unifying the representation of log events. Through the semantic feature extraction module, the model can effectively distinguish between different log events, maintaining good differentiation even when the log format is unstable or data changes occur.

2.3 Anomaly Detection

This article combines CNN, Bi-GRU, and attention mechanism together. Build a log anomaly detection module CB, as shown in Figure 1.

Input the extracted statistical feature vectors and semantic feature vectors into the CB module for processing. After merging the output results, input them into a fully connected layer for linear transformation to obtain the predicted results.

Firstly, input the obtained feature vectors into the CNN layer. Extract local patterns and feature information through convolutional kernels. In the log sequence, adjacent logs have a certain correlation, so this paper adopts one-dimensional convolution 1D-CNN [9] to learn the patterns inside the log sequence. 1D-CNN only performs one-dimensional convolution, namely scalar multiplication and addition, which is suitable for extracting features from fixed length fragments. Use one-dimensional convolution kernels to scan the embedding vectors of log sequences and solve the problem of dimensional imbalance by filling in zeros at both ends. After completion, use the feature maps extracted by CNN as input and pass them to the Bi GRU layer.

Bi-GRU inputs the input sequence into two gated neural networks with the same network structure, forward and backward. Capture the temporal dependencies between the current moment and previous time steps. At each time step t, concatenate the hidden states of the forward GRU and the reverse GRU,

$$\overrightarrow{h_t} = \overrightarrow{z_t} \odot \overrightarrow{h_{t-1}} + \left(1 - \overrightarrow{z_t}\right) \odot \overrightarrow{x_t} \tag{6}$$

$$\overleftarrow{h_t} = \overleftarrow{z_t} \odot \overleftarrow{h_{t+1}} + \left(1 - \overleftarrow{z_t}\right) \odot \overleftarrow{x_t} \tag{7}$$

$$h_t = \overrightarrow{h_t} \odot \overleftarrow{h_t} \tag{8}$$

h_t (Formula 6,7.8) represents the hidden vector of the current state, z_t is the update gate, x_t is the candidate state, $\odot$ is the concatenation operation.

The importance of hidden states at different time steps for anomaly detection varies. Introduce attention mechanism, calculate attention scores et (Formula 9) for different time steps of the input sequence, dynamically allocate weights a_t (Formula 10),

$$e_t = v^t \tanh\left(W h_t + b\right) \tag{9}$$

$$a_t = \frac{\exp\left(e_t\right)}{\sum_{t=1}^{T} \exp\left(e_t\right)} \tag{10}$$

$$y = W_C C + b_c \tag{11}$$

Where W is the weight matrix, v is the weight vector, and b is the bias term. W_c and b_c are fully connected layer learning parameters.

Then, attention weights are used to weight and average the hidden states of all time steps, generating a context vector c that summarizes the key information of the entire input sequence. Finally, it is transformed into a single output y (Formula 11) through a fully connected layer. After activating the function, output the model prediction result. If it is greater than the set threshold, it is an abnormal log.

3 Experimental results and analysis

3.1 Experimental environment parameters and dataset

The experiment uses Python 3.7 and PyTorch 1.2. The experimental platform is Windows 11, Intel i5-7400, 16GB of memory, and Nvidia 4060Ti graphics card. Experimental parameter settings: CNN input channel 300, output channel 250, convolution step size reduced, Bi-GRU hidden layer size of 128, 2 layers, anomaly detection classifier trained using Adma optimizer, loss calculated using cross entropy

Table 1: Evaluation Index Information of the Model.

Method	HDFS			BGL		
	Pr	Re	F1	Pr	Re	F1
OES	0.98	0.96	0.97	0.93	0.98	0.95
LogBERT	0.94	0.92	0.93	0.93	0.9	0.91
LogAnomaly	0.96	0.94	0.95	0.97	0.94	0.96
LogGD	0.98	0.96	0.97	0.97	0.96	0.96
LogCGA	0.99	0.97	0.98	0.99	0.97	0.98

loss function. The epoch is 380, the initial learning rate is 0.001, the batch size is 64, and the dropout dropout probability is 0.1. The dataset is selected from publicly available datasets HDFS and BGL. The HDFS dataset comes from a distributed operating system. The BGL dataset comes from supercomputer system logs.

3.2 Evaluation indicators

Using precision Pr(Formula 12), recall Re(Formula 13), and F1(Formula 14) as evaluation indicators: precision refers to the ratio of correctly identified abnormal log sequences to all sequences identified as abnormal by the model; The recall rate is the ratio of correctly identified abnormal log sequences to all actual anomalies; F1 is the harmonic mean of precision and recall. Their calculation formulas are as follows

$$pr = TP/(TP + FP) \tag{12}$$

$$Re = TP/(TP + FN) \tag{13}$$

$$F1 = 2 \times Pr \times Re/(Pr + Re) \tag{14}$$

TP represents the number of cases judged as abnormal by the model, FN represents the number of cases judged as normal by the model, and FP represents the number of cases misjudged by the model.

3.3 Experimental result

This article selects four methods as benchmark models. They are OES[10] and LogAnomaly, as well as LogGD[5] and LogBERT.Compare with the LogCGA model proposed in this article. The results are shown in Table 1.

From Table 1, it can be seen that on the HDFS and BGL datasets, the accuracy and recall of the benchmark model remain above 0.9, while the accuracy and recall of the LogCGA model reach above 0.98, which is superior to other benchmark models. This proves that the model can maintain excellent performance on different log data.

In order to further test the detection performance of the LogCGA model on unstable datasets, this paper randomly selects logs from the log sequence to create an unstable log sequence. After the experiment began, unstable data was added to the test data at a ratio of 5%, 10%, 15%, and 20%. The test results of the five methods on the HDFS dataset are shown in Figure 2.

The results in Figure 2 show that as the proportion of unstable logs increases, the F1 values of all methods decrease. Although the LogCGA model proposed in this article has also decreased, the F1 value remains above 0.9. Prove that this model has high robustness against unstable logs. The model can better adapt to situations where logs undergo changes.

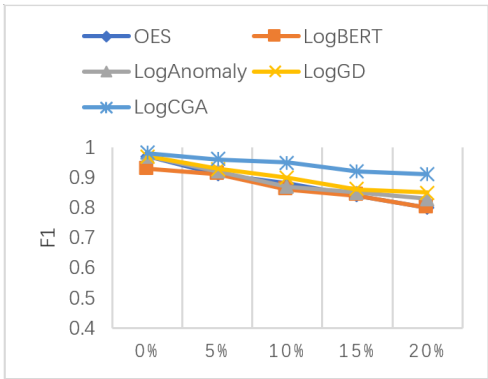

Figure 2: Evaluation of the effectiveness of unstable logs.

3.4 Ablation experiment

In order to evaluate the impact of each module on the experimental results, three ablation experiments were conducted on the HDFS dataset. Model A removed the semantic feature extraction module from LogCGA; The B model removes logarithmic count vectors from the statistical feature extraction module LogCGA; The C model removed the 1D-CNN layer from the anomaly detection algorithm. As shown in Table 2, semantic feature extraction improved the F1 score performance of the model by 4.2 percentage points, indicating that learning semantic features can enhance the robustness of the model. Logarithmic statistical vectors and 1D CNN increased the F1 score of the model by 2.1 and 1.4 percentage points, respectively. The results of the ablation experiment indicate that all three modules have a certain effect on anomaly detection.

4 Conclusion

This article proposes a log anomaly detection model LogCGA, which addresses the issue of reduced accuracy in anomaly detection caused by unfamiliar logs. The FastText language model is used to extract semantic features of words in the logs, and the TF-IWF algorithm is used to obtain weighted sentence embedding vectors, enabling the model to learn key semantic features. Effectively improved the robustness of the model. By combining CNN and Bi-GRU neural networks, local features of logs are learned and temporal dependencies in data are captured. Attention mechanisms are used to highlight more important features for anomaly detection, increasing the accuracy of anomaly detection. The experimental results indicate that LogCGA performs better than existing methods in anomaly detection tasks. The focus of this article is on detecting abnormal data in logs. However, no explanation has been provided for the reasons behind the prediction results. In future research, we will explore the interpretability of the anomaly detection process from different perspectives. Explore using alternative methods for log detection to improve detection efficiency.

Table 2: Results of ablation experiment.

Model	Pr	Re	F1
LogCGA	0.996	0.983	0.989
A	0.965	0.93	0.947
B	0.974	0.964	0.968
C	0.98	0.971	0.975

References

[1] LE V H, ZHANG H. Log-based anomaly detection with deep learning: how far are we?[C]// Proceedings of the 44th International Conference on Software Engineering. New York: ACM, 2022, 1356-1367.

[2] YANG L, CHEN J, WANG Z, *et al.* Semi-supervised log-based anomaly detection via probabilistic label estimation [C]// Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering. Piscataway: IEEE, 2021, 1448-1460.

[3] LI B, MA S, DENG R, *et al.* Federated anomaly detection on system logs for the internet of things: a customizable and communication-efficient approach [J]. IEEE Transactions on Network and Service Management, 2022, 19(2): 1705-1716.

[4] DUAN X, YING S, YUAN W, *et al.* QLLog: a log anomaly detection method based on Q-learning algorithm[J]. Information Processing and Management, 2021, 58(3): No. 102540.

[5] XIE Y, ZHANG H, BABAR M A. LogGD: Detecting Anomalies from System Logs with Graph Neural Networks[C]//2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS). Guangzhou, China: IEEE, 2022, 299-310.

[6] DAI H, LI H, CHEN C S, *et al.* Logram: efficient log parsing using n-gram dictionaries[J]. IEEE Transactions on Software Engineering, 2022, 48(3): 879-892.

[7] TAO S, MENG W, CHENG Y, *et al.* LogStamp: automatic online log parsing based on sequence labelling[J]. ACM SIGMETRICS Performance Evaluation Review, 2022, 49(4): 93-98.

[8] YANG L, CHEN J, WANG Z, *et al.* Semi-supervised log-based anomaly detection via probabilistic label estimation [C]// Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering. Piscataway: IEEE, 2021, 1448-1460.

[9] KIRANYAZ S, AVCI O, ABDELJABER O, *et al.* 1D convolutional neural networks and applications: a survey[J]. Mechanical Systems and Signal Processing, 2021, 151: No. 107398.

[10] HAN S, WU Q, ZHANG H, *et al.* Log-based anomaly detection with robust feature extraction and online learning[J]. IEEE Transactions on Information Forensics and Security, 2021, 16: 2300-2311.